

Department of Computer Science and Engineering

Remote Lab: Orgazination Description

50565: Ângelo Filipe Maia Azevedo (a50565@alunos.isel.pt)
50539: António Miguel Alves (a50539@alunos.isel.pt)

Advisor: Prof. Pedro Miguens Matutino

July 2025

Project Structure

The Remote Lab project is organized into several main directories, most of which are managed as GitHub submodules. This modular approach enables independent development, versioning, and access control for each core component, supporting both scalability and security. Submodules also facilitate collaboration among different teams and ensure that sensitive information is handled appropriately.

The main submodules and directories are:

- **api/** – Contains the backend source code (Kotlin, Spring Boot). Responsible for business logic, user management, and laboratory session control.
- **db/** – Includes database scripts and documentation, supporting the system's persistence layer.
- **website/** – Holds the frontend web application (Next.js, React). Provides the user interface for laboratory access, scheduling, and management.
- **docker/** – Contains Docker-related scripts and files for deployment and automation.
- **nginx/** – Provides NGINX configuration files for reverse proxying, with separate configs for development and production (`default.dev.conf`, `default.prod.conf`).
- **docs/** – Stores project documentation, including technical reports, user guides, and architectural diagrams.
- **img/** – Contains project images, such as diagrams, screenshots, and other visual assets.
- **private/** – Dedicated to sensitive files and configurations, such as environment variables and secrets. This submodule is not included directly in the main repository, ensuring that only authorized members have access to confidential information like API (Application Programming Interface) keys and external service credentials. Shared secrets are located in `private/shared/secrets/.env`.
- **wiki/** – Stores the GitHub Wiki pages, including project documentation, deployment instructions, and other relevant information.
- **hardware/** – Contains code and configuration for integrating and simulating hardware devices, enabling interaction with real or mock laboratory equipment.

This structure allows for independent development, testing, and deployment of each component. The addition of the `docker/` directory and the clear separation of NGINX configs reflect the evolution towards more robust DevOps and automation practices.

Repository Access

The project repository is available here.

To run the application with the full configuration, access to the `private/` submodule is required, as it contains essential API keys and secret files. This submodule is private for security reasons; therefore, contributors must request access to the “Remote Lab” GitHub organization or directly to the private repository to obtain the necessary credentials and configuration files.

Both the backend (`api/`) and frontend (`website/`) repositories maintain separate `main` and `develop` branches to support stable releases and ongoing development, respectively. This branching strategy facilitates collaborative development and continuous integration while ensuring production stability.

How to Run the Project

To run the Remote Lab platform, simply use the provided `start.sh` script at the project root. This script automatically loads all required environment variables and secrets from the `private/` submodule, ensuring a correct and secure configuration for all services.

1. Clone the Repository and Submodules:

Clone the main repository and initialize all submodules, including `private/`, to ensure all components and configurations are available.

```
git clone --recurse-submodules
https://github.com/isel-remote-lab/remote-lab.git
cd remote-lab
```

2. Run the Platform:

Start all services using the automation script:

```
./start.sh -dev
```

The script will build and start all necessary containers, and automatically load environment variables and secrets from `private/`.

3. Access the Platform:

By default, the platform will be accessible at `http://localhost:80`. If the Cloudflare Tunnel option is enabled (using the appropriate flag in `start.sh`), remote access will also be available via the Cloudflare-provided public URL.

Notes:

- The **private/** submodule is required for full configuration, as it contains essential API keys and secret files.
- The **start.sh** script supports several flags for different deployment scenarios (development, production, demo, cloudflare, etc.).
- For secure remote access, use the Cloudflare Tunnel option in **start.sh**.
- **For more information about available options and usage examples, run `./start.sh` without arguments. The script will display detailed instructions and usage examples.**